

coop-deberta

Архитектура обучения и инференса ко-оп бота
и история его версий

Состояние репозитория на 4 октября 2026 г. · бот по умолчанию — v31 (`coop-deberta-v3-ens3-v31`)

24

приказа: 23 действия и
NONE («не приказ»)

3 × 184 млн

параметров: ансамбль из
трёх DeBERTa-v3-base

≈ 40 мс

на одну реплику на трёх
ядрах CPU

2.3 ГБ

оперативной памяти, GPU
в игре не нужен

О чём этот документ. coop-deberta — языковой слой напарника-бота в тактическом шутере. Он получает английскую реплику игрока в виде текста и возвращает один приказ из закрытого списка, место на карте и два признака: «исполнить по моему сигналу» и «взять другой объект». Документ объясняет, как эта система обучается, что происходит с репликой во время игры и как система менялась от версии к версии.

Для кого. Для разработчика, который не работал с этим проектом. Раздел 0 с нуля объясняет основы: классификатор, softmax, BERT и DeBERTa; кто с ними знаком, может начать с раздела 1. Термины проекта объяснены в разделе 2. Все цифры взяты из документов и логов репозитория: `HANDOFF.md`, `COOP-BOT.md`, `DEBERTA-BOT.md`, `cpp/coop_intent/README.md` и кода в `scripts/coop`.

Содержание

0. Основы: классификатор, softmax, BERT и DeBERTa	2
1. Что такое coop-deberta	6
2. Словарь терминов и список приказов	8
3. Модель: DeBERTa-v3-base с головой классификации	8
4. Данные для обучения	9
5. Архитектура обучения	11
рецепт · одно обучение · боевая модель <code>final_ens3</code> · порог и гейт · кросс-валидация · экспорт	
6. Архитектура инференса	16
путь реплики · слоты · токенизатор · ансамбль · гейт · логика решения · места · Python и C++ · скорость	
7. История версий	22
предыстория · v1 · v2 · v3 · v31 · сводные таблицы	
8. Уроки проекта	25
9. Шпаргалка: команды и карта файлов	26

0. Основы: классификатор, softmax, BERT и DeBERTa

Этот раздел — короткий курс по четырём понятиям, на которых держится всё остальное. Каждое объяснено с нуля и сразу привязано к coop-deberta. Кто с ними знаком, может перейти к разделу 1.

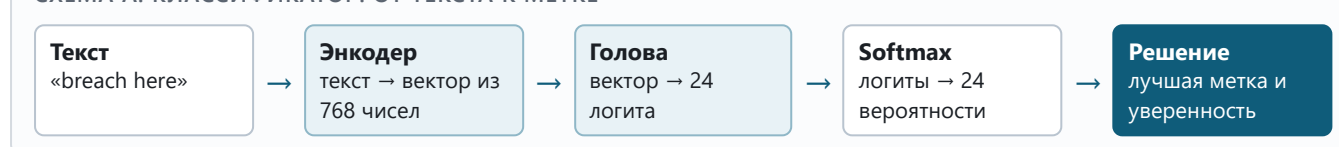
0.1. Классификатор текста

Классификатор — это модель, которая получает текст и выбирает для него одну метку из заранее заданного закрытого списка. Фильтр спама выбирает между «спам» и «не спам». coop-deberta выбирает между 24 приказами.

В отличие от генеративной модели вроде чат-бота, классификатор не пишет текст. Отсюда три его свойства:

- **Ответ всегда из списка.** Игровой логике не нужно разбирать свободный текст: она получает номер метки.
- **Он быстрый.** На одну реплику нужен один проход по сети, без пошаговой генерации слов.
- **У ответа есть числовая уверенность.** По ней можно решить, действовать или переспросить.

СХЕМА А. КЛАССИФИКАТОР: ОТ ТЕКСТА К МЕТКЕ



Из чего он состоит

- **Энкодер** превращает текст в вектор — набор чисел фиксированной длины, в котором закодирован смысл реплики. В coop-deberta это 768 чисел, а энкодер — DeBERTa (разделы 0.3 и 0.4).
- **Голова классификации** — один линейный слой. Он умножает вектор на матрицу весов и прибавляет смещение: $z = W \cdot h + b$. На выходе получается по одному числу на метку. Эти числа называются **логитами**.

Логит — необработанная оценка «за» метку: чем он больше, тем сильнее модель склоняется к этой метке. Логит может быть отрицательным и сам по себе вероятностью не является. В вероятности логиты превращает softmax (раздел 0.2).

Масштаб. Голова coop-deberta — это матрица 24×768 и 24 смещения: 18 456 чисел. Во всей модели 184 миллиона параметров, так что почти вся работа происходит в энкодере. Голова только читает готовый вектор.

Как он учится

Классификатору показывают размеченные примеры: текст и правильную метку. Для каждого примера считается **функция потерь** — число, которое тем больше, чем меньше модель верила в правильную метку. Затем веса сдвигаются так, чтобы потери уменьшились. Это повторяется много раз: у coop-deberta 1520 шагов на 1213 примерах.

Дообучение (fine-tuning) означает, что энкодер берут готовый, уже обученный языку на огромном корпусе текстов, а голову создают заново со случайными весами. Потом на своих примерах учат и то, и другое. Поэтому хватает сотен примеров, а не миллионов.

Чего классификатор не умеет

- **Сказать «не знаю».** Такого выхода у него нет: он всегда выбирает из списка. Поэтому в проекте есть метка `NONE` («это не приказ») и порог уверенности.
- **Выдать метку, которой нет в списке.** Новый приказ требует новых примеров и переобучения. Так в версии v2 появились `TAKE_COVER` и `OPEN`.

0.2. Softmax

Softmax — функция, которая превращает набор логитов в набор вероятностей: все числа становятся положительными, а их сумма равна единице.

$$p_i = e^{z_i} / (e^{z_1} + e^{z_2} + \dots + e^{z_K})$$

Здесь z_i — логит метки i , K — число меток, $e \approx 2.718$. Словами: каждый логит возводится в экспоненту, и результат делится на сумму всех экспонент.

Пример с четырьмя метками

Метка	Логит z	e^z	Вероятность p
BREACH	4.0	54.598	0.853
ENTRY	2.0	7.389	0.115
SMOKE	0.5	1.649	0.026
NONE	-1.0	0.368	0.006
Сумма		64.004	1.000

Вероятность BREACH: $54.598 / 64.004 = 0.853$. Отрицательный логит тоже даёт положительную вероятность.

Четыре свойства, которые стоит помнить

1. **Порядок сохраняется.** У самого большого логита самая большая вероятность. Чтобы узнать лучшую метку, softmax не нужен; он нужен, чтобы узнать уверенность.
2. **Важна разница логитов, а не их величина.** Если ко всем логитам прибавить одно и то же число, вероятности не изменятся.
3. **Экспонента усиливает разрыв.** Разница логитов в 2 единицы даёт отношение вероятностей $e^2 \approx 7.4$: в примере это 0.853 против 0.115.
4. **Сумма всегда равна единице.** Модель обязана распределить всю уверенность между метками, даже если текст не похож ни на одну из них.

Вероятность softmax — это не шанс оказаться правым. Модель, которую долго учили на небольшом наборе, уверена почти всегда: 0.95 не означает «права в 95% случаев». А из-за четвёртого свойства она уверенно отвечает и на то, чего никогда не видела: на русскую реплику «прикрой меня» бот v1 ответил REVIVE_ME с вероятностью 0.92. Поэтому порог уверенности нельзя назначать на глаз: его подбирают на строках, которых модель при обучении не видела (раздел 5.5).

Softmax при обучении

Функция потерь классификатора — **кросс-энтропия**: минус логарифм вероятности, которую модель дала правильной метке. Если в примере выше правильная метка — BREACH, потери равны $-\ln 0.853 = 0.16$: модель почти права, и веса почти не меняются. Если правильная метка — ENTRY, потери равны $-\ln 0.115 = 2.16$: веса сдвигаются сильно, логит ENTRY растёт, остальные падают.

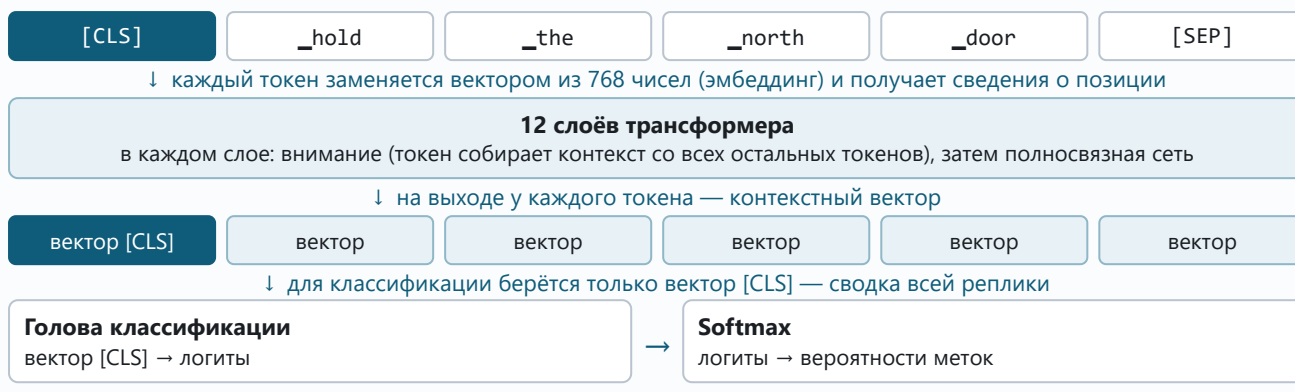
Где softmax в coop-deberta

- Каждая из трёх моделей выдаёт 24 логита, и softmax превращает их в 24 вероятности.
- Три набора вероятностей усредняются. Усредняются именно вероятности, а не логиты: это обычный способ объединять несколько моделей.
- Гейт family складывает вероятности приказов одного семейства (раздел 6.5). Складывать их можно потому, что все 24 вероятности — доли одной единицы.
- Softmax работает и внутри энкодера: им считаются веса внимания (раздел 0.3).

0.3. BERT

BERT (Bidirectional Encoder Representations from Transformers, Google, 2018) — энкодер на архитектуре «трансформер». Он читает весь текст сразу и для каждого токена выдаёт вектор, в котором учтён контекст и слева, и справа. BERT не пишет текст, он его понимает, и поэтому хорошо подходит для классификации.

СХЕМА Б. КАК ЭНКОДЕР ЧИТАЕТ РЕПЛИКУ «HOLD THE NORTH DOOR»



Что происходит внутри

- Токенизация.** Текст режется на токены — слова или куски слов из фиксированного словаря, — и каждый token заменяется своим номером. В начало ставится служебный token [CLS], в конец — [SEP].
- Эмбединги.** Каждому номеру соответствует строка в таблице эмбедингов: вектор из 768 чисел. Модели нужны ещё и сведения о позиции токена, иначе она не отличила бы «dog bites man» от «man bites dog».
- Слои трансформера.** Векторы проходят через 12 одинаково устроенных слоёв. В каждом два шага: внимание и полносвязная сеть.
- Выход.** После последнего слоя у каждого токена есть контекстный вектор. Вектор токена [CLS] используют как сводку всего текста: именно его получает голова классификации.

Внимание

Внимание (self-attention) — механизм, которым токены обмениваются информацией. Для каждого токена модель считает веса: насколько для него важен каждый другой token текста. Затем новый вектор токена собирается как взвешенная смесь векторов всех токенов. Веса считаются через softmax, поэтому в сумме дают единицу.

В реплике «hold the other angle» вектор слова «angle» после внимания несёт в себе и «other», и «hold»: по нему уже видно, что речь о другом угле, который надо удерживать. В каждом слое 12 «голов» внимания работают параллельно и ловят разные связи между словами.

Почему «двунаправленный»

Генеративные модели семейства GPT читают текст слева направо: при обработке слова они видят только то, что было до него. BERT видит слова с обеих сторон сразу. Для короткой реплики это важно: слово «hold» в «hold on» означает «подожди», а в «hold the north door» — «держи дверь», и различить их можно только по словам справа.

Предобучение и дообучение

- Предобучение.** На огромном корпусе текстов без разметки (у BERT это книги и английская Википедия) модель решает задачу, для которой разметка не нужна: в тексте скрывают около 15% токенов, и модель восстанавливает их по контексту. Задача называется masked language modeling (MLM). Чтобы её решать, модели приходится выучить грамматику и значения слов.
- Дообучение.** К предобученному энкодеру добавляют голову под свою задачу и учат на своих размеченных примерах. Язык модель уже знает, ей остаётся выучить только саму задачу.

Модель	Слоёв	Размер вектора	Голов внимания	Параметров
BERT-base	12	768	12	110 млн
BERT-large	24	1024	16	340 млн

0.4. DeBERTa и DeBERTa-v3

DeBERTa (Decoding-enhanced BERT with disentangled attention, Microsoft, 2020) — развитие BERT. Общая схема та же: токены, эмбединги, слои с вниманием, вектор `[CLS]`. Изменилось то, как модель учитывает позиции слов, а в третьей версии — и то, на какой задаче она предобучается.

Распутанное внимание

В BERT вектор содержания токена и вектор его позиции складываются в один ещё на входе, и дальше модель работает с этой смесью. В DeBERTa они хранятся отдельно: у токена есть вектор содержания, а позиция задаётся относительно — на сколько токенов левее или правее стоит другой токен. Вес внимания между токенами i и j складывается из трёх частей (упрощённо):

$$\text{вес}(i, j) = \text{содержание}_i \cdot \text{содержание}_j + \text{содержание}_i \cdot \text{позиция}_{i \rightarrow j} + \text{позиция}_{j \rightarrow i} \cdot \text{содержание}_j$$

Первое слагаемое отвечает на вопрос «подходят ли эти слова друг другу по смыслу». Второе и третье — «важно ли для этого слова то, что стоит на таком расстоянии от него». Связь между словами зависит и от того, какие это слова, и от того, как далеко они друг от друга: «other» прямо перед «angle» значит не то же, что «other» через пять слов. Раздельные векторы позволяют модели выучить это напрямую.

Второе изменение — «улучшенный декодер масок», от которого в названии слово Decoding-enhanced: абсолютная позиция токена добавляется только в самом конце, перед предсказанием скрытого слова. Оно касается предобучения; при классификации этот декодер не используется.

DeBERTa-v3: другая задача предобучения

В третьей версии (2021) задачу «восстанови скрытое слово» заменили задачей «найди подмену» (replaced token detection, взята у модели ELECTRA). Маленькая вспомогательная модель-генератор заменяет часть слов текста правдоподобными вариантами, а основная модель для каждого токена отвечает, оригинал это или подмена.

Выгода в том, что учебный сигнал идёт от каждого токена текста, а не только от 15% скрытых: на том же количестве текста модель учится большему. Именно эта версия стоит в имени наших моделей:

`coop-deberta-v3-...`

	BERT-base	DeBERTa-v3-base
Год и автор	2018, Google	2021, Microsoft
Слои, размер вектора, головы	12, 768, 12	12, 768, 12
Параметры	110 млн	184 млн
Словарь	около 30 тысяч кусков слов	128 тысяч кусков слов
Позиции слов	Абсолютные; складываются с содержанием на входе	Относительные; отдельный вектор в каждом слое внимания
Задача предобучения	Восстановить скрытые слова (MLM)	Найти подменённые слова (RTD)

DeBERTa-v3-base в цифрах

Числа посчитаны по файлу весов модели v31 (`seed0/model.safetensors`).

Блок модели	Параметров	Доля
Таблица эмбедингов: 128 100 токенов × 768 чисел	98.4 млн	53.3%
12 слоёв трансформера	85.1 млн	46.1%
Голова: промежуточный слой 768 × 768 и классификатор 24 × 768	0.6 млн	0.3%
Векторы относительных позиций и нормализация эмбедингов	0.4 млн	0.2%
Всего	184.4 млн	100%

Больше половины модели — таблица эмбедингов: словарь DeBERTa-v3 вчетверо больше, чем у BERT. При работе из этой таблицы читается лишь несколько строк, по одной на токен реплики. Поэтому она занимает память, но на скорость почти не влияет: время определяют 12 слоёв. Те же свойства видны в файле `config.json` модели:

```
"num_hidden_layers": 12, "hidden_size": 768, "num_attention_heads": 12 # размеры как у BERT-base
"vocab_size": 128100 # размер словаря
"relative_attention": true, "pos_att_type": ["p2c", "c2p"] # распутанное внимание
"position_biased_input": false # позиции на вход не подмешиваются
```

Как четыре понятия складываются в coop-deberta

Понятие	Что это в coop-deberta
Энкодер	DeBERTa-v3-base: превращает реплику игрока в вектор [CLS] из 768 чисел
Классификатор	Голова поверх энкодера: 24 логита, по одному на приказ
Softmax	Превращает логиты в вероятности; вероятности трёх моделей усредняются
Дообучение	Предобученный энкодер и новая голова учатся на 1213 размеченных репликах (раздел 5)
Чего не хватает	Softmax не умеет говорить «не знаю», поэтому поверх стоят гейт, порог и метка NONE (раздел 6)

1. Что такое coop-deberta

Игрок в шутере в духе Rainbow Six Siege или Counter-Strike отдаёт напарнику-боту голосовые приказы на английском: «breach here», «hold the north door», «smoke the west window on my go». Распознавание речи превращает голос в строку текста. Задача coop-deberta — превратить эту строку в то, что может исполнить игровая логика.

На каждую реплику система выдаёт:

- **приказ** — одну из 24 меток (BREACH , MOVE_TO , ..., NONE);
- **решение, действовать ли вообще**: исполнить, поставить в очередь, переспросить или промолчать;
- **слоты**: приказ ждёт сигнала игрока; приказ относится к «другому» объекту;
- **места на карте**, названные в реплике, и то из них, которое является целью бота.

Как именно исполнить приказ — маршрут, укрытие, какая из дверей ближайшая, — решает планировщик игры. Языковой слой в это не вмешивается.

Какие ошибки дорогие

Владелец игры заранее определил цену ошибок, и вся архитектура построена вокруг этого решения.

Ситуация	Оценка	Пример
Перепутаны приказы одного семейства	Терпимо: бот «буквальный и слегка туповатый», и это принято	BREACH вместо ENTRY : оба — штурм
Действие из чужого семейства	Недопустимо	Бот бросает гранату вместо того, чтобы держать угол
Действие на реплику, которая приказом не была	Недопустимо	«two pushing through B apps» — сообщение о противнике, а бот идёт на штурм
Преждевременное действие	Недопустимо	Приказ отдан «on my go», а бот исполняет сразу
Отказ	Нормальный ответ: в игре это реплика «Say again?»	Модель не уверена — бот переспрашивает

СХЕМА 1. ПУТЬ РЕПЛИКИ ОТ ГОЛОСА ДО ПЛАНИРОВЩИКА



Три механизма, и только один из них — нейросеть

Часть	Что решает	Чем сделана
Классификатор	Какой это приказ	Ансамбль из трёх дообученных DeBERTa-v3-base, softmax по 24 меткам
Слоты	«По моему сигналу», «другой», отрицание в начале реплики	Три регулярных выражения
Места	Куда: дверь, окно или лестница, сторона или цвет, этаж; чьё это место	Словарь и позиционные правила (сопоставитель мест)
Логика бота	Действовать, ждать сигнала, переспросить или промолчать	Обычный код: гейт, порог, очередь из одного отложенного приказа

Почему не всё делает модель. Модификатор «on my go» — это два-три слова. Модели размазывают такой признак по всему предложению: на тестовых строках они слышали сигнал либо почти везде, либо почти нигде, а регулярное выражение не придумало сигнал ни в одной из 288 строк. Названия мест — закрытый список, одинаковый на всех картах, и он будет пополняться: словарь принимает новое название сразу, без переобучения. Правило проекта: то, что ловится простым кодом, отдаётся коду.

Как читать имя модели. В имени coop-deberta-v3-ens3-v31 часть deberta-v3 — это архитектура (третья версия DeBERTa), а не версия бота. ens3 — ансамбль из трёх моделей. Версия бота стоит в конце: v31. Самый первый бот называется coop-deberta-v3-base: одна модель, без суффикса версии.

2. Словарь терминов и список приказов

Термин	Что означает в проекте
Приказ (intent)	Одна из 24 меток классификатора. <code>NONE</code> — «это не приказ»: сообщение о противнике, болтовня, вопрос или отменённый приказ.
Семейство	Группа приказов, ошибка внутри которой терпима (штурм, гранаты, удержание и так далее).
Попадание (near)	Выбор бота принят двумя из трёх читателей строки или лежит в том же семействе, что и большинство.
Чужое семейство (wrong family)	Бот действовал (выбрал не <code>NONE</code>), и его выбор не принят и не входит в нужное семейство. Главная ошибка.
Действие на не-приказ	Доля строк с истинной меткой <code>NONE</code> , на которых бот выбрал что-то другое.
Критерий	Попадание – 2 × чужое семейство. Двойка кодирует решение владельца: ошибка вдвое хуже отказа. По критерию выбираются порог и гейт.
Порог	Минимальная уверенность, при которой бот действует. Ниже порога он переспрашивает.
Гейт	Способ посчитать уверенность: <code>top</code> — вероятность лучшего приказа; <code>family</code> — суммарная вероятность лучшего семейства.
Seed-команды	Опорный словарь: короткие канонические команды («come here», «breach»), написанные разработчиком. Идут только в обучение.
Слепые строки	Реплики, написанные авторами-агентами, которые видели только спецификацию приказов. На них измеряется качество.
Out-of-fold (OOF)	Предсказание для строки, сделанное моделью, которая эту строку при обучении не видела.
Коллаут	Короткое сообщение игрока о противнике или состоянии места: «two on red stairs», «north door is smoked». Это не приказ.
Планировщик	Игровая логика, которая исполняет приказ. В проект не входит.

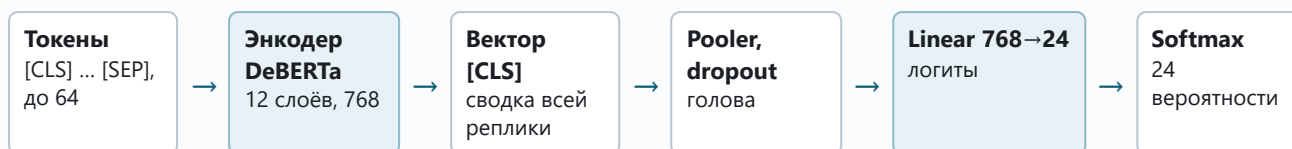
24 приказа по семействам

Семейство	Приказы	Смысл
<code>assault</code>	<code>BREACH</code> , <code>ENTRY</code> , <code>VAULT_WINDOW</code> , <code>RAPPEL</code>	Штурм: взорвать, войти и зачистить, влезть в окно, спуститься по верёвке
<code>utility</code>	<code>FLASH</code> , <code>SMOKE</code> , <code>FRAG</code>	Гранаты: светошумовая, дымовая, боевая
<code>hold</code>	<code>HOLD_POSITION</code> , <code>HOLD_ANGLE</code> , <code>HOLD_OTHER_ANGLE</code> , <code>COVER_ME</code>	Стоять на месте, держать угол, держать другой угол, прикрыть игрока
<code>move</code>	<code>FOLLOW_ME</code> , <code>MOVE_TO</code> , <code>FLANK</code>	Идти за игроком, идти к месту, обойти
<code>objective</code>	<code>PLANT</code> , <code>DEFUSE</code>	Поставить или обезвредить заряд
свои семейства	<code>DRONE</code> , <code>REVIVE_ME</code> , <code>FALL_BACK</code> , <code>WAIT</code> , <code>GO_NOW</code> , <code>TAKE_COVER</code> , <code>OPEN</code> , <code>NONE</code>	Каждый приказ — отдельное семейство: путать его ни с чем нельзя. <code>TAKE_COVER</code> (бот прячется сам) и <code>OPEN</code> (открыть, не входя) добавлены в v2

3. Модель: DeBERTa-v3-base с головой классификации

DeBERTa-v3-base — энкодер семейства BERT: 12 слоёв трансформера, размер скрытого состояния 768, 184 млн параметров. Как он устроен и чем отличается от BERT, объяснено в разделе 0.4. Класс `AutoModelForSequenceClassification` добавляет к энкодеру голову классификации. Каждая из трёх моделей ансамбля — один и тот же предобученный энкодер `microsoft/deberta-v3-base`, дообученный на данных проекта с разным начальным сидом (0, 1, 2). Одна модель в fp32 занимает 738 МБ.

СХЕМА 2. ОДНА МОДЕЛЬ АНСАМБЛЯ



Метки живут в весах

Последний линейный слой имеет 24 выхода, по одному на приказ: список приказов зашит в веса. Это «классический» классификатор. Альтернатива, которую проект тоже измерял, — модели, получающие варианты ответа текстом во входе (laya, decision:eos).

	Метки в весах (coop-deberta)	Варианты во входе (laya, decision:eos)
Новый приказ	Нужны размеченные примеры и переобучение	Достаточно дописать вариант в меню
Скорость на CPU	Около 40 мс у ансамбля; 26–40 мс у одной модели	220–310 мс у дообученной laya; 2.5 с у decision:eos
Строгий порог	Хорошо отсекает свои ошибки: при 2% чужого семейства сохраняет 82.8% попаданий (одна модель, исследование v1)	Дообученная laya слишком уверена в ошибках: при тех же 2% сохраняет 74.4%
Без размеченных данных	Не работает	decision:eos работает сразу, но только на GPU (244 мс)

Что пробовали вместо этого и почему отказались

- **ModernBERT-base и -large.** Даже после отдельного подбора гиперпараметров уступают: на новых формулировках DeBERTa-v3-base впереди ModernBERT-base на 8.7 пункта (интервал [+2.0, +16.8]).
- **deberta-v3-large.** На фиксированном рецепте один сид из трёх развалился на двух проверках из трёх и отвечал `NONE` почти на всё. Там, где large обучилась нормально, она не лучше base, а в бюджет памяти рядом с ансамблем не помещается.
- **Квантизация в int8** (`quantize_dynamic`). Сломала модель: лучший приказ сохранился только на 85 из 592 проверочных строк. В игре работает fp32.
- **fp16 на CPU.** Медленнее, а не быстрее: 384 мс против 67–71 мс в fp32.

Зачем ансамбль

Три модели с разными сидами ошибаются по-разному, и среднее их вероятностей ровнее. В исследовании v2 ансамбль против одной модели дал +5.6 пункта критерия на всех строках (интервал [+2.8, +8.4]) и +9.5 на новых формулировках ([+4.8, +14.1]). Цена — память: 2.3 ГБ вместо 0.8 ГБ. По времени ансамбль на трёх ядрах стоит столько же, сколько одна модель на одном потоке: около 40 мс.

4. Данные для обучения

4.1. Слепые строки

Записей настоящих игроков у проекта нет. Их заменяют строки, написанные ИИ-агентами в трёх ролях. Каждый автор видел только спецификацию приказов (`blind/spec_v*.json`) и квоту по видам строк. Меток других авторов и результатов тестов он не видел — отсюда слово «слепые».

Код	Роль автора	Особенность строк
r6	Игрок Rainbow Six Siege	Плотный сленг
cs	Игрок CS2	Лексика Counter-Strike
stt	Вывод распознавания речи	Ошибки распознавания («beach» вместо breach, «repel» вместо rappel), речь неносителей языка

Партия	Строк	Что в ней
v1	3 × 121 = 363	21 приказ и NONE : по 4 строки на приказ, приказы «по сигналу», «другой», коллауты, болтовня, отрицания, составные приказы
v2	3 × 40 = 120	TAKE_COVER , OPEN и приказы, с которыми их легко спутать
v3	3 × 50 = 150	Приказы и коллауты, в которых названы места на карте
Всего	633	На них обучены v3 и v31

Истина для каждой строки — большинство из трёх прочтений: метка автора и метки двух независимых аннотаторов. Кроме основной метки хранится множество допустимых приказов (accept): те, которые приняли хотя бы двое.

Чему в этих данных нельзя верить.

- Авторы и аннотаторы — агенты одного семейства моделей, и они повторяют друг друга: у 163 из 363 строк v1 есть близкая копия у другого автора, 22 совпадают дословно. Поэтому каждая цифра отдельно считается на «новых» строках, у которых нет близкой копии в обучении (косинус по символьным n-граммам ниже 0.5).
- Разметчики единодушны на 98% строк. У людей так не бывает.
- Приказы сбалансированы, а в игре «go» звучит в сотни раз чаще, чем «rappel».
- Настоящего распознавания речи в данных нет: автор stt его имитирует.

4.2. Seed-команды

Первый живой тест модели, обученной только на слепых строках, провалился: «come here» → WAIT 0.48, «go there» → GO_NOW 0.39, голое «breach» → DEFUSE 0.37. Порог спас от глупостей — бот переспрашивал, — но пользы от него не было. Причина в том, как писались данные: авторам было велено не повторять слова из описаний приказов, чтобы тест проверял устойчивость к перефразированию. Для теста это правильно, для обучения — нет: самых простых и частых формулировок в данных почти не оказалось.

Решение — seed-команды: короткие канонические команды, написанные разработчиком. Они добавляются только в обучение и никогда не оцениваются как тестовые строки. На кросс-валидации попадание выросло с 83.3 до 89.8, и прирост сохранился на 155 строках, далёких и от seed-команд, и от строк других авторов (68.8 → 81.9 при равном риске). Значит, seed-команды учат смыслу приказа, а не конкретным словам.

Набор	Команд	Что добавлено к предыдущему
seed_commands.json (v1)	204	По 8 команд на приказ, не-приказы и отрицания («don't follow me», «don't breach yet»)
seed_commands_v2.json	225	16 команд и 5 отрицаний для TAKE_COVER и OPEN
seed_commands_v21.json	233	8 команд COVER_ME с голым «cover» («cover while I move»)
seed_commands_v3.json	383	150 шаблонных команд с местами: 50 шаблонов приказа, каждый заполнен тремя случайными местами
seed_commands_v31.json	580	Две команды про крышу перемечены в MOVE_TO; 192 шаблонные команды (каждый шаблон — с местами каждого вида; коллауты о состоянии в NONE); 5 коллаутов без места

4.3. Переключатель COOP_TAG

Переменная окружения `COOP_TAG` выбирает набор строк и набор seed-команд. От неё зависят имена файлов результатов и папка, в которую сохраняется модель (`coop-deberta-v3-ens3-<тег>`). Так прежние исследования остаются воспроизводимыми.

COOP_TAG	Строк	Seed-команд	Для чего
не задан	483	225	Исследование v2
v21	483	233	Боевой бот v2
v3	633	383	Исследование v3 и финальная модель v3
v31	633	580	Модель v31. Строки автора r6 взяты с независимым перечитыванием разметки

4.4. Заморозка

Спецификация, seed-команды и правила сопоставителя мест фиксируются **до** появления тестовых строк. Их хэши (sha256) и порядок событий записаны в `scripts/coop/frozen.txt` . Если что-то правится после того, как тестовые строки прочитаны, это записывается там же, а цифра помечается «после подгонки». Читатель отчёта всегда знает, какая цифра слепая, а какая нет.

5. Архитектура обучения

Обучение устроено в три слоя. Нижний — одно обучение одной модели по фиксированному рецепту. Средний — сборка боевого бота: три модели и подбор порога и гейта. Верхний — оценка: кросс-валидация по авторам, которая отвечает на вопрос, чего ждать от бота на незнакомых игроках. Весь код лежит в `scripts/coop/train_v2.py` ; данные, гейты и подсчёт метрик — в `coop_v2.py` .

5.1. Рецепт

Гиперпараметры выбраны один раз, до первых результатов, и не менялись с версии v1 (`train_bert.HP`). Отдельный вложенный перебор показал, что DeBERTa от подбора ничего измеримого не получает: +1.9 и +2.8 пункта, оба интервала включают ноль.

Параметр	Значение	Зачем
Основа	<code>microsoft/deberta-v3-base</code> , 24 выхода	Веса и токенизатор берутся с HF Hub
Точность	<code>float32</code> , задаётся явно	Чекпойнт сохранён в <code>fp16</code> : см. ловушку ниже
Оптимизатор	<code>AdamW</code> , <code>weight decay</code> 0.01	Лёгкая регуляризация; на CUDA используется fused-вариант (тот же алгоритм)
Скорость обучения	<code>5e-5</code>	Стандарт для base-энкодеров
Расписание	10% шагов линейный разогрев, затем линейный спад	Стабилизирует начало, пока голова ещё случайная
Эпохи	20	Данных мало; перебор показал, что модели нужно 18–20 эпох, а ограничение в 10 ей вредит
Батч	16	Маленький набор — больше шагов обновления
Длина	до 64 токенов	Реплики короткие: в среднем 8.4 токена
Функция потерь	кросс-энтропия по 24 классам	
Сиды	0, 1, 2	На малых данных результат заметно зависит от сйда

Ловушка: fp16 и NaN. Библиотека transformers 5 загружает чекпойнт в том типе, в котором он сохранён, а deberta-v3-base сохранён в fp16. В fp16 параметр eps оптимизатора AdamW (1e-8) округляется до нуля, и после первого же шага все веса становятся NaN: модель отвечает одним классом на всё. Поймали это по тому, что все три сйда дали одинаковые 11.6%. Поэтому при загрузке везде стоит dtype=torch.float32.

5.2. Одно обучение

Функция train_predict обучает одну модель и возвращает её вероятности на проверочных строках:

```
model = AutoModelForSequenceClassification.from_pretrained(
    "microsoft/deberta-v3-base", num_labels=24, dtype=torch.float32).to(device)
steps = epochs * ceil(len(train) / 16)
opt = torch.optim.AdamW(model.parameters(), lr=5e-5, weight_decay=0.01)
sched = get_linear_schedule_with_warmup(opt, int(0.1 * steps), steps)
for _ in range(epochs):                                # 20 эпох
    random.shuffle(order)
    for idx in batches(order, 16):
        loss = model(**train_batch(idx), labels=labels[idx]).loss # кросс-энтропия
        loss.backward(); opt.step(); sched.step(); opt.zero_grad()
```

Для v31 обучающих строк 1213 (633 строки исследования и 580 seed-команд). Это 76 батчей в эпохе и 1520 шагов оптимизатора, из них первые 152 — разогрев. Каждая строка токенизируется один раз и лежит на устройстве; батч дополняется до самой длинной строки в нём.

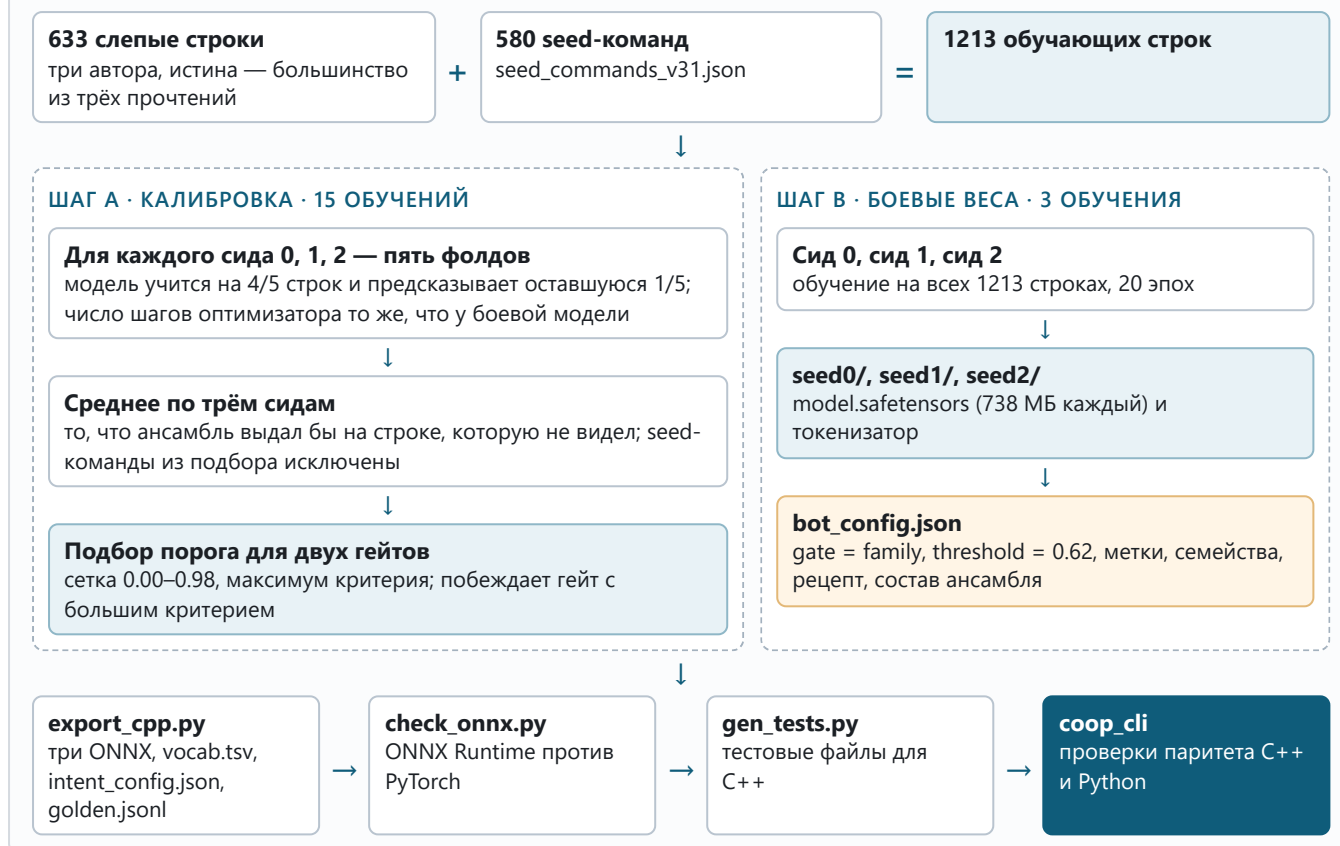
5.3. Три режима train_v2.py

Команда	Что делает	Обучений
train_v2.py smoke	Проверяет, что устройство работает: 64 строки, 2 эпохи, вероятности суммируются в единицу	1, около минуты
train_v2.py final ens3	Собирает боевого бота: порог, гейт и три набора весов	18
train_v2.py cv base	Исследование: кросс-валидация по авторам	54

5.4. Боевая модель: final ens3

Для игры данные не прячут: три боевые модели учатся на всех строках. Но порог и гейт нельзя подбирать на строках, которые модель видела, поэтому перед финальными обучениями делается калибровка на out-of-fold предсказаниях.

СХЕМА 3. КОМАНДА COOP_TAG=v31 python train_v2.py final ens3 — 18 ОБУЧЕНИЙ



5.5. Откуда берутся порог и гейт

Softmax выдаёт числа, похожие на вероятности, но откалиброванными они не являются. На строках, которые модель видела при обучении, она уверена во всём, и порог по ним вышел бы бесполезным. Поэтому используется out-of-fold.

- Пять фолдов.** Все 1213 строк перемешиваются и делятся на пять частей. Для каждой части модель учится на остальных четырёх и предсказывает эту часть.
- То же число шагов.** У фолдовой модели данных на пятую часть меньше, поэтому эпох ей дают больше, чтобы число шагов оптимизатора совпало с боевой моделью. Для v31 это 25 эпох вместо 20.
- Три сида.** Процедура повторяется для сидов 0, 1 и 2, и вероятности трёх сидов усредняются. Получается ответ ансамбля на строке, которой ни одна из его моделей не видела.
- Без seed-команд.** Порог подбирается только по строкам исследования: seed-команды слишком простые, и порог по ним вышел бы заниженным.
- Подбор.** Для каждого гейта перебирается порог от 0.00 до 0.98 с шагом 0.02. Выбирается порог с наибольшим критерием, посчитанным в строках: число попаданий – 2 × число действий из чужого семейства. При равенстве берётся меньший порог.
- Выбор гейта.** Побеждает гейт с большим критерием; при равенстве остаётся более простой `top`.

Модель	Строк	top: порог / критерий	family: порог / критерий	Выбрано
v2	483	0.48 / 429	0.58 / 432	family, 0.58
v3 (обучена на MacBook)	633	0.60 / 560	0.60 / 569	family, 0.60
v31	633	0.62 / 561	0.62 / 566	family, 0.62

Ловушка: порог на недообученных моделях. В первой версии протокола порог подбирался на моделях, обученных на одном авторе, то есть на вдвое меньших данных и за вдвое меньше шагов. Такие модели неуверенные: порог вышел 0.32 при оптимальных около 0.54, и бот действовал там, где должен был переспросить. Правило: порог подбирают на моделях, обученных так же, как та, что пойдёт в игру.

Сокращённый вариант — флаг `--reuse-gate <путь к bot_config.json>`: делаются только три финальных обучения, а гейт и порог берутся у другой модели и под новый набор данных не подбираются.

5.6. Оценка качества: кросс-валидация по авторам

Режим `cv` отвечает на вопрос, как бот поведёт себя с игроками, чьей речи он не слышал. Модель учится на строках двух авторов и всех `seed`-командах, а проверяется на третьем авторе. Так три раза.

Прогон	Обучение	Проверка
1	строки <code>cs</code> + строки <code>stt</code> + <code>seed</code> -команды	строки <code>r6</code>
2	строки <code>r6</code> + строки <code>stt</code> + <code>seed</code> -команды	строки <code>cs</code>
3	строки <code>r6</code> + строки <code>cs</code> + <code>seed</code> -команды	строки <code>stt</code>

Для каждого прогона и каждого из трёх сидов делается одно обучение на проверку и пять `out-of-fold` обучений для порога этого прогона: $3 \times 3 \times 6 = 54$ обучения. Ансамбль в оценке — это среднее вероятностей трёх сидов; отдельного обучения он не требует.

Почему по авторам, а не случайно. При случайном разбиении почти-дубли одного автора попали бы и в обучение, и в тест, и цифры бы ввали. Разбиение должно имитировать ситуацию в игре, а туда придут новые игроки со своей манерой речи.

Три правила чтения цифр, которые проект выучил на своих ошибках:

- **Метрики под задачу.** Считаются попадание, чужое семейство, действие на не-приказ и критерий, а не общая точность.
- **Сравнение при одинаковом риске.** Модель без порога действует всегда: попаданий у неё много, но и ошибок много. Сравнивать её с моделью с порогом нельзя.
- **Доверительные интервалы.** Рядом с каждой разницей стоит интервал парного бутстрэпа по строкам. На 150–200 строках разница в несколько пунктов — шум.

5.7. Устройства и время

Устройство выбирается автоматически: `cuda`, иначе `mps` (Apple silicon), иначе `cpu`. Переменная `COOP_DEVICE` задаёт его явно.

Что обучалось	Где	Время
v3, <code>final ens3</code> (18 обучений)	MacBook, Apple M5 Pro, MPS	53 мин 20 с; около 178 с на обучение; пик памяти 9.0 ГБ
v31, <code>final ens3</code> (18 обучений)	RTX 4090, CUDA	44 мин параллельно с двумя потоками кросс-валидации
v31, кросс-валидация (54 обучения)	RTX 4090, CUDA	Три процесса, по одному на автора

Другое устройство — другая модель. Прямой проход на MPS совпадает с CPU (расхождение до $1.2e-5$), но обучение на разных устройствах даёт разные веса. У v3 таких моделей две: обученная на MacBook (порог 0.60, она в репозитории) и обученная на CUDA (порог 0.52, в репозиторий не попала). Повторное обучение под тем же тегом на другой машине даст другую модель под тем же именем.

5.8. Что сохраняется

```
models/coop-deberta-v3-ens3-v31/
├─ bot_config.json          метки, семейства, гейт, порог, рецепт, на чём обучена, результаты подбора
├─ seed0/ seed1/ seed2/    model.safetensors (738 МБ, fp32), config.json, tokenizer.json
└─ cpp/                   всё для C++-движка, пишет export_cpp.py
    ├─ model0.onnx model1.onnx model2.onnx    по 739 МБ, fp32, opset 17
    ├─ vocab.tsv            словарь Unigram: «кусоч, табуляция, оценка»; номер строки — id токена
    ├─ intent_config.json  метки, семейства, порог, гейт, три регэкспа, словарь мест
    ├─ golden.jsonl        1259 строк: текст, id токенов, вероятности PyTorch
    └─ *_tests.jsonl, dialogue.jsonl, bench.txt  тесты паритета и замеры скорости
```

Весов в git нет: файлы по 738 МБ не проходят лимит GitHub в 100 МБ. В `.gitignore` стоят `*.safetensors` и `*.onnx`; веса переносятся между машинами отдельно.

5.9. Экспорт и три ступени проверки

1. `export_cpp.py` переводит каждую модель в ONNX: входы `input_ids` и `attention_mask`, выход — логиты; размер батча и длина строки динамические; opset 17 (его поддерживает NNE в UE5). Он же пишет словарь токенизатора, конфигурацию и golden-файл.
2. `check_onnx.py` сверяет ONNX Runtime с PyTorch до всякого C++: тот же лучший приказ и вероятности в пределах примерно $1e-4$. Если после этого C++ разойдётся с Python, виноват может быть только порт.
3. `gen_tests.py` прогоняет Python-бота и записывает тестовые файлы для C++: токенизатор, места, гейты, решения и диалог.

Ловушка экспорта: включённый dropout. `torch.onnx.export` после экспорта возвращает модуль-обёртку в исходный режим, а свежий модуль находится в режиме обучения. Модель осталась с включённым dropout, и первый golden-файл оказался шумом. Нашлось это так: ONNX Runtime точно совпал со свежим прогоном PyTorch, но не с golden-файлом. Теперь после экспорта стоит явный `model.eval()`.

6. Архитектура инференса

6.1. Путь одной реплики

Разберём реплику «smoke the west window on my go» при боте v31 (схема 1 в разделе 1). Числа в таблице — настоящий вывод бота.

№	Шаг	Что происходит	Время
1	Слоты	Три регэкспа читают сырую строку. ON_SIGNAL находит «on my go»; OTHER и NEGATION не срабатывают	микросекунды
2	Токенизация	Строка превращается в 9 токенов: [CLS] _smoke _the _west _window _on _my _go [SEP]	≈ 20 мкс
3	Три модели	Каждая выдаёт 24 логита; softmax превращает их в вероятности; три распределения усредняются: SMOKE 0.999	≈ 40 мс
4	Гейт и порог	Вероятности складываются по семействам. Семейство utility набирает массу 0.999 — выше порога 0.62; его лучший приказ — SMOKE	микросекунды
5	Места	Сопоставитель находит одно место: объект window, уточнение west. Оно и есть основная цель	2–5 мкс
6	Решение	Приказ принят, и сработал слот «по сигналу»: приказ ставится в очередь вместе с местом. Бот отвечает «Ready to throw a smoke. On your go.»	микросекунды

Следующая реплика «go go go» распознаётся как GO_NOW (0.995), и бот исполняет отложенный SMOKE у западного окна. Почти всё время уходит на шаг 3; остальное — микросекунды.

6.2. Слоты: три регулярных выражения

```
ON_SIGNAL  \b(on|at) (my|the) (go|call|mark|signal|count|word|command)\b
           | \bwhen i (say|call|give|tell|shout)
           | \bon (one|two|three|1|2|3)\b(?: of\b)          # не «on one of the windows»
           | \b(wait|hold) (for|till|until) (my|i)\b
           | \b(till|until) i say
OTHER      \b(other|another|opposite)\b
NEGATION   ^\W*(?:(:uh|um|so|okay|ok|hey|bot|buddy|please|nah|no)\W+)*
           (don't|do not|dont|never|no need to)\b          # одна строка; перенос сделан для печати
```

Слот	Что значит для бота	Качество на тестовых строках v1
ON_SIGNAL	Приказ не исполняется сразу, а ждёт сигнала игрока	Найдено 21 из 24 приказов «по сигналу»; ни одного ложного на 288 обычных приказах
OTHER	Планировщик берёт другой объект того же вида	Найдено 26 из 30; ни одного ложного на 282 строках
NEGATION	Реплика начинается с отрицания: приказ отменяется	Правило «отрицание в начале» на слепых строках ничего не стоит

Отрицание ловится только в начале реплики. Правило «отрицание где угодно» проверили и отвергли: оно стоило 22 пункта на строках с отрицанием, потому что настоящие приказы часто содержат его в середине («hold that door, don't let anyone through»). Голое «но» в правило не входит: «no, the other door» — это приказ.

Цифры по слотам — верхние оценки: спецификация, которую видели авторы строк, сама называла фразы вроде «on three».

6.3. Токенизатор

Модель обучена с быстрым токенизатором DebertaV2 из библиотеки HF tokenizers. Это не обычный конвейер sentencepiece, и его шаги важно знать тому, кто переносит модель на другой язык:

1. **Добавленные токены.** [CLS], [SEP], [PAD], [UNK], [MASK], набранные прямо в тексте, распознаются первыми как служебные.
2. **Нормализация.** Каждая серия пробелов и каждый перевод строки или табуляция заменяются одним пробелом; затем NFC; пробелы справа обрезаются.
3. **Metaspace.** Пробелы заменяются символом «`_`» (U+2581), он же ставится в начало; каждое «`_`» начинает новое слово.
4. **Unigram.** Для каждого слова выбирается разбиение на куски словаря с наибольшей суммарной оценкой (алгоритм Витерби). Символ, которого нет в словаре, становится [UNK].
5. **Сборка.** [CLS] + куски + [SEP]; куски обрезаются до 62, чтобы вместе со служебными токенами было не больше 64.

В Python это делает библиотека. В C++ конвейер воспроизведён шаг за шагом (`tokenizer.cpp`, `unicode.cpp`), без ICU и без вызовов операционной системы, и сверен с Python по id токенов.

6.4. Ансамбль: усредняются вероятности

Каждая модель переводит свои логиты в вероятности через softmax, и усредняются уже вероятности, а не логиты: $p = (p_0 + p_1 + p_2) / 3$. В Python это одна строка: `torch.stack([softmax(m(**x).logits[0]) for m in models]).mean(0)`. В C++ класс `EnsembleBackend` считает softmax в двойной точности, усредняет и возвращает логарифм среднего: softmax от него даёт то же среднее, поэтому остальной код не отличает ансамбль от одной модели.

6.5. Гейт и порог

Гейт отвечает на вопрос «насколько модель уверена», порог — «достаточно ли этого, чтобы действовать».

- **Гейт top:** уверенность — вероятность лучшего приказа.
- **Гейт family:** вероятности складываются по семействам; берётся семейство с наибольшей массой и его лучший приказ; уверенность — масса семейства.

Пример, ради которого гейт `family` создан (числа из исследования v2). На реплику «break in» модель ответила: BREACH 0.51, ENTRY 0.47, всё остальное 0.02.

Гейт	Уверенность	Решение при пороге 0.62
top	0.51 — вероятность BREACH	Ниже порога: бот переспросит
family	0.51 + 0.47 = 0.98 — масса семейства assault	Выше порога: бот исполнит BREACH

Модель не может выбрать между двумя приказами одного семейства, но ошибка между ними терпима, поэтому отказ здесь был бы лишним. Для приказов со своим семейством (например, OPEN) оба гейта дают одно и то же. Бот v31 на ту же реплику отвечает увереннее: ENTRY 0.82, BREACH 0.18, масса семейства 0.99.

Порог — ручка политики микрофона. При разговоре по кнопке порог можно ослабить. Открытому микрофону нужен строгий край, потому что большая часть услышанного — коллауты. В чате бота порог меняется командой `/t 0.6`; значение по умолчанию берётся из `bot_config.json`.

6.6. Логика решения

Функция `Bot.decide` в Python и её двойник `BotBrain::Decide` в C++ получают текст и вероятности и возвращают действие. Проверки идут сверху вниз, срабатывает первая подходящая.

№	Условие	Действие	Что делает бот
1	Реплика начинается с отрицания, а выбранный приказ не <code>NONE</code> , <code>WAIT</code> или <code>HOLD_POSITION</code>	<code>negated</code>	«Copy, standing down.» Отложенный приказ сбрасывается
2	Уверенность ниже порога, или вместо вероятностей пришёл NaN	<code>say_again</code>	Переспрашивает и ничего не делает
3	Приказ — <code>NONE</code>	<code>ignore</code>	Подтверждает («Copy.») и ничего не делает. Места из реплики передаются как контакты
4	Приказ — <code>GO_NOW</code>	<code>execute</code> или <code>go</code>	Если в очереди есть приказ — исполняет его с его местом и его слотом «other». Если очереди нет — «Going!»
5	Приказ — <code>WAIT</code>	<code>wait</code>	Сбрасывает отложенный приказ вместе с местом
6	Сработал слот <code>ON_SIGNAL</code>	<code>queued</code>	Ставит приказ в очередь вместе с местами и слотом «other»
7	Всё остальное	<code>act</code>	Исполняет приказ

- **Очередь — одна ячейка.** Новый приказ «on my go» заменяет прежний.
- **NaN безопасен.** Условие записано как «не (уверенность $\geq$ порога)», поэтому NaN всегда даёт «say again».
- **Отложенный приказ помнит своё место.** При исполнении используются место и слот «other» отложенного приказа, а не реплики «go». Иначе «smoke the west window on my go» → «go» дымило бы неизвестно куда.
- **Журнал.** Каждая реплика пишется в `coop_bot_log.jsonl`. В настоящей игре это данные для следующей разметки.

```

you> smoke the west window on my go
bot> Ready to throw a smoke. On your go.      ← SMOKE 0.999; слот «по сигналу»: приказ в очередь
      place: * west window                    ← основная цель
you> go go go
bot> Now! Smoke out.                          ← GO_NOW 0.995: исполнен отложенный SMOKE у west window
you> two on red stairs
bot> Noted.                                   ← NONE 1.000: коллаут; red stairs [them] — контакт
you> get off the roof
bot> Didn't catch that.                       ← MOVE_TO 0.603, FALL_BACK 0.350: разные семейства
      place: * roof [from]                   ← 0.603 < 0.62: переспрос; roof — место, откуда уйти

```

Настоящие ответы и числа Python-бота v31 на CPU. Отладочная строка бота заменена пояснениями справа.

6.7. Места на карте

Названия мест в классификатор не добавлялись: приказов по-прежнему 24. Места находит сопоставитель — словарь `locations.json` и позиционные правила (`locations.py`, двойник в C++ — `locations.cpp`). Сопоставитель не зависит от модели и работает с любой версией бота.

Раздел словаря	Содержимое	Примеры слов
Объекты (6)	<code>door</code> , <code>window</code> , <code>stairs</code> , <code>chair</code> , <code>sofa</code> , <code>table</code>	«doorway», «couch», «stares» (ошибка распознавания)
Уточнения (10)	<code>main</code> ; четыре стороны света; пять цветов	Двери и окна различаются по сторонам света, лестницы — по цветам
Зоны (7)	<code>basement</code> , <code>floor_1</code> , <code>floor_2</code> , <code>top_floor</code> , <code>roof</code> , <code>up</code> , <code>down</code>	«ground floor» — это <code>floor_1</code> ; «upstairs» — этаж выше бота
Именованные фразы (8)	Объект и уточнение одной фразой	«front door» → <code>door</code> / <code>main</code>

Одиночное уточнение тоже может быть местом: «take blue» означает синюю лестницу, «push main» — главную дверь.

Запись для планировщика

На каждую реплику сопоставитель возвращает запись: все места по порядку, роль каждого и номер основной цели. Для реплики «i'll take the north door, you take the south window»:

Цель (объект / уточнение)	Роль	Основная цель
door / north	mine — это место игрока	
window / south	нет	да: сюда идёт бот

Роль или помета	Значение	Пример
роль mine	Место игрока	«i'll take the north door»
роль them	Место противника; для планировщика это контакт	«two on red stairs»
роль from	Место, откуда надо уйти	«get off the roof»
роль not	Отрицание или поправка	«not the north door, the south one»
роль status	О месте только сообщают	«north door is clear, hold the south window»
помета unknown_modifier	Игрок выделил один объект способом, для которого у карты нет названия. Ближайший объект брать нельзя	«the back door», «the door on the left»
помета other	Другой объект того же вида	«take the other door»
помета unsure	Одиночное название перед незнакомым словом: уверенность низкая, но чаще всего это настоящее место	«the white fence»

Договор с планировщиком:

- Цель с ролью — не место назначения. Основная цель — первая цель без роли.
- Если роль есть у всех целей, основной считается первая, и это значит, что места назначения в реплике нет. Планировщик обязан проверить роль основной цели.
- При NONE места всё равно возвращаются: коллаут «two on red stairs» даёт планировщику контакт.
- Бот отдаёт приказ и место независимо и не сверяет их между собой.

Технически сопоставитель работает по байтам UTF-8: слово — это серия латинских букв и цифр, всё остальное — разделитель. Он читает последние 128 слов реплики, выбрасывает слова-паузы («uh», «um») и не использует ни регэкспов, ни таблиц Unicode. Если меняется только словарь мест, правила или регэкспы, переобучать модель не нужно (раздел 9).

6.8. Две реализации: Python и C++

Python-бот (scripts/coop/coop_bot.py) — эталон: PyTorch на CPU и чат в терминале. **C++-движок** (cpp/coop_intent) — то, что пойдёт в игру: C++17 и ONNX Runtime. Он проверен построчно против Python-бота и разложен так, чтобы ядро переехало в UE5 без изменений.

Файл C++	Что делает	Двойник в Python	В UE5
unicode.cpp	UTF-8, NFC и классы символов из сгенерированных таблиц	библиотека tokenizers	как есть
tokenizer.cpp	Токенизатор DebertaV2	AutoTokenizer	как есть
intent_model.cpp	Интерфейс ILogitsBackend, ансамбль, softmax	Bot.classify	как есть

Файл C++	Что делает	Двойник в Python	В UE5
<code>ort_backend.cpp</code>	Одна модель в ONNX Runtime 1.30	модель PyTorch	заменяется бэкэндом NNE
<code>bot_brain.cpp</code>	Гейт, порог, защита от отрицания, очередь, слоты	<code>Bot.pick</code> , <code>Bot.decide</code>	как есть; регэкспы через ICU
<code>locations.cpp</code>	Сопоставитель мест	<code>locations.py</code>	как есть
<code>tools/coop_cli.cpp</code>	Чат в терминале и все проверки	<code>coop_bot.py</code>	не переносится

Модель спрятана за интерфейсом `ILogitsBackend` с одним методом: на входе id токенов одной реплики, на выходе логиты. Сейчас его реализует `OrtBackend`; порт на UE5 реализует тот же метод через NNE. Сессия ONNX Runtime настроена так: заданное число потоков внутри операции, один поток между операциями, полная оптимизация графа; маска внимания состоит из единиц, потому что реплика всегда одна и дополнять её нечем.

6.9. Потоки, скорость и память

Как работают три модели	Потоков на модель	Потоков всего	Медиана	99-й процентиль
Параллельно, каждая на своём потоке	4	12	35.4 мс	43.7 мс
Параллельно	2	6	33.3 мс	42.1 мс
Параллельно — настройка для игры	1	3	39.4 мс	54.0 мс
По очереди	4	4	57.0 мс	74.7 мс
По очереди	2	2	63.1 мс	90.7 мс
По очереди	1	1	87.0 мс	140.6 мс

Процессор i9-10900K, fp32, ONNX Runtime на CPU. Таблица измерена на ансамбле v3 той же архитектуры. Бот v31: 40.8 мс при одном потоке на модель и 56.4 мс по очереди с четырьмя потоками.

- **Память и загрузка.** Три модели занимают 2.3 ГБ и загружаются за 4.5 с. Одна модель v1 — 0.8 ГБ и 2.0 с.
- **Всё, кроме моделей, почти бесплатно.** Токенизатор — около 20 мкс на реплику, сопоставитель мест — 2–5 мкс.
- **Настройка для игры** — по одному потоку на модель: три ядра примерно на 40 мс. Флаг `--no-spin` (рабочие потоки спят между вызовами) ничего измеримого не стоит.
- **Python-бот** на PyTorch медленнее: 67–71 мс на одной модели v1.
- **План для UE5.** Токенизатор и `BotBrain` работают в игровом потоке; модели — на рабочих потоках, результат возвращается в игровой поток. `BotBrain` хранит очередь, поэтому живёт в одном потоке. Модели загружаются асинхронно при старте уровня.

6.10. Паритет Python и C++

C++-движок считается верным только тогда, когда построчно совпадает с Python-ботом. Проверки запускает `coop_cli`; ниже результаты для v31 на рабочей машине.

Флаг <code>coop_cli</code>	Что сравнивается	Результат v31	Модель
<code>--golden</code>	Id токенов и вероятности против PyTorch	1259 из 1259, расхождение до 2.2e-6	нужна
<code>--tokenizer-tests</code>	Нормализатор и id токенов; три регэкспа	9428 из 9428; регэкспы 9424 из 9424	не нужна
<code>--location-tests</code>	Записи мест: цели, роли, пометы, основная цель	20204 из 20204	не нужна
<code>--gate-tests</code>	Оба гейта на строках вероятностей	3332 из 3332	не нужна

Флаг <code>coop_cli</code>	Что сравнивается	Результат v31	Модель
<code>--decide-tests</code>	Все ветки решения и поведение очереди	140 из 140	не нужна
<code>--dialogue</code>	Разговор из 77 реплик через модели	77 из 77	нужна

Что эти проверки поймали при переносе:

- **Таблицы Unicode.** Библиотека `tokenizers` нормализует текст по более старым таблицам Unicode, чем модуль `unicodedata` в Python. Модель обучена на том, что выдаёт `tokenizers`, поэтому таблицы для C++ сгенерированы из её поведения.
- **Якорь `^` в MSVC.** `std::regex` в MSVC считает началом строки позицию после каждого перевода строки, а `re` в Python — только начало текста. Ведущий `^` поэтому выполняется как поиск строго с начала, а любой другой якорь отвергается при загрузке.
- **Предел рекурсии.** `std::regex` в MSVC сдаётся после примерно 118 слов-пауз перед отрицанием. В этом случае слот считается отсутствующим, программа не падает; четыре тестовые строки проверяют именно это.

6.11. Известные пределы

- **Только английский.** Реплика «прикрой меня» дала `REVIVE_ME` 0.92, и порог это не ловит (замечено на v1). Неанглийские реплики надо отсеивать до модели.
- **Регистр.** Модель различает регистр и видела в основном строчные реплики. На `seed`-командах, набранных заглавными буквами, v31 действует из чужого семейства на 12 из 233.
- **Один приказ на реплику.** В составном приказе («smoke it and push») выполняется только одно действие.
- **«on two», «on three»** читаются как счёт сигнала, даже когда игрок говорит об этаже.
- **Места вне словаря** не находятся: «second story», «rough» вместо roof.
- **Настоящее распознавание речи** не измерялось; планировщик и разрешение целей на карте не построены.

7. История версий

- 27 сентября 2026. Исследование без дообучения: семь готовых моделей на 363 слепых строках.
- 28 сентября. **v1**: одна дообученная DeBERTa-v3-base, 22 приказа. Живой тест, seed-команды, защита от отрицания.
- 29 сентября. **v2**: приказы `TAKE_COVER` и `OPEN`, ансамбль из трёх моделей, гейт по семейству. C++-движок.
- 3–4 октября. **v3**: места на карте. Сопоставитель мест и дообучение на строках с местами. Ревью находит, что модель исполняет коллауты.
- 4 октября. **v31**: исправленный набор seed-команд. Владелец делает v31 ботом по умолчанию.

до v1 Какая модель вообще годится

27 сентября 2026

До DeBERTa проект сравнил семь готовых моделей без дообучения на 363 слепых строках. Из этого сравнения выросли три решения, которые дожили до текущей версии.

- **Без размеченных данных работает только decision:eos** с описаниями приказов и порогом 0.52: 62% попаданий при 2.2% действий из чужого семейства. Но ей нужен GPU: 244 мс на GPU и 2.5 с на CPU.
- **Слоты лучше любой модели ловит регэксп.** Так появились `ON_SIGNAL` и `OTHER`.
- **Для игры только на CPU быстрых и безопасных вариантов без меток нет.** Лучший из них действовал из чужого семейства на 5.8% строк при 189 мс. Отсюда решение дообучать свою модель.

v1 Одна дообученная DeBERTa-v3-base

28 сентября 2026

models/coop-deberta-v3-base · 22 приказа · 363 строки и 204 seed-команды · одна модель · гейт top, порог 0.78

- **Первый результат.** Кросс-валидация по авторам без seed-команд: попадание 75.5%, чужое семейство 4.8%, действие на не-приказ 13.7%. На CPU 67 мс вместо 189 мс у лучшей готовой модели, пригодной для CPU.
- **Сравнение с ModernBERT.** Каждой модели подобрали свой рецепт; DeBERTa-v3-base осталась впереди.
- **Первый живой тест** показал, что бот не понимает «come here» и голое «breach». Добавили 204 seed-команды: попадание без порога выросло с 83.3 до 89.8. На боевом пороге 0.78 бот даёт 84.3% попаданий при 2.5% чужого семейства.
- **Вторая находка живого теста:** «do not come to me» → `FOLLOW_ME` 0.93. Модель читает тему, а не полярность. Исправили дважды: отрицания в seed-командах и правило отрицания в начале реплики.
- **Параллельно дообучили laua** — модель с вариантами во входе. Качество близкое, меню остаётся гибким, но она в 3–4 раза медленнее и хуже отсекает свои ошибки порогом.

v2 «Спрячься», «открой» и ансамбль из трёх моделей

29 сентября 2026

models/coop-deberta-v3-ens3-v2 · 24 приказа · 483 строки и 233 seed-команды (набор v21) · три модели · гейт family, порог 0.58

Зачем. В живом тесте владельца бот путал «прикрой меня» и «спрячься сам», а «open the door» превращал в `BREACH` с уверенностью 0.99 — бот взорвал бы дверь. Увеличение модели тут не помогает: у классификатора просто не было нужных меток.

- **Два новых приказа**, `TAKE_COVER` и `OPEN`, каждый в своём семействе. На 120 новых строках старый бот давал 41.7% попаданий и 25.8% действий из чужого семейства.
- **120 новых слепых строк.** Старые 363 перечитали под новую спецификацию: ни одна не получила новый приказ.
- **Три кандидата** в бюджете владельца (до 200 мс на реплику и до 2 Гб памяти сверх прежних 0.8 Гб): одна base, одна large, ансамбль из трёх base. Large оказалась нестабильной, ансамбль победил.

- **Гейт по семейству** появился в этой версии.
- **Набор seed-команд v21.** После исследования выяснилось, что «cover from behind» (так, с опечаткой) уходит в TAKE_COVER : бот прячется вместо того, чтобы прикрыть. Добавили 8 команд COVER_ME с голым «cover». Фразы владельца починились, но действий из чужого семейства на боевой настройке стало 9 строк вместо 5 из 483 (в пределах шума: $-1.2 [-4.1, +1.7]$).
- **Результат** на кросс-валидации: попадание 90.5%, чужое семейство 1.9%, действие на не-приказ 5.3%.
- **C++-движок** на ONNX Runtime и проверки паритета появились тогда же.

Путаница в названиях. Исследование называется v2, набор seed-команд, на котором обучен боевой бот, — v21, а папка модели — ...-ens3-v2 . Все три названия относятся к одному боту.

models/coop-deberta-v3-ens3-v3 · 24 приказа · 633 строки и 383 seed-команды · три модели · гейт family, порог 0.60 (обучена на MacBook, MPS) · ботом по умолчанию не была

Зачем. На всех картах игры одни и те же названные места, и игроки называют их и в приказах («hold the north door»), и в коллаутах («two on red stairs»).

- **Главное решение: названия не идут в модель.** Их находит сопоставитель мест (раздел 6.7). Новое название — строка в словаре, без переобучения.
- **Правила сопоставителя прошли три редакции** (таблица ниже). Слепая цифра есть только у первой.
- **150 новых слепых строк** с местами и 150 шаблонных seed-команд.
- **Дообучение помогло на строках с местами:** попадание 83.3 → 90.7, чужое семейство 3.3% → 1.3%. Почти весь выигрыш дал один автор (r6 : 78 → 98), чьи новые строки написаны с заглавными буквами и пунктуацией. На 483 старых строках изменений не найдено.
- **Вырезать названия мест перед классификатором не помогло:** меняются 3 ответа из 150, все в худшую сторону. Модель читает строку как есть.

Правила сопоставителя	Точная цель, 150 строк	Пригодная цель, 123 приказа	Статус цифры
v1, заморожены до появления строк	88.0%	83.7%	слепая
v2, после чтения строк двух авторов	96.7%	91.1%	после подгонки
v3, после двух ревью	98.0%	99.2%	после подгонки

Пригодная цель — найдена точно и не несёт ни роли, ни пометы «не уверен»: планировщик может по ней действовать.

Почему v3 не стала ботом по умолчанию. Два ревью (73 находки на одной машине и 47 на другой) пробами нашли то, чего слепые строки не показали, потому что таких реплик в них не было:

- коллаут о дыме выполняется как приказ: «<место> is smoked» → SMOKE на 47 названиях из 47 (у бота v2 — на 7);
- «<место> clear» → ENTRY на 27 названиях из 30;
- «get on the roof» → RAPPEL : seed-команды, унаследованные от v2, противоречили новой спецификации, по которой идти на крышу без верёвки — это MOVE_TO ;
- «take blue» → ENTRY : в шаблонные seed-команды попали только формы со словом «stairs».

По тем же ревью исправили движок: отложенный приказ теперь хранит своё место и слот «other», NaN даёт «say again», загрузчик отвергает повреждённую конфигурацию. Правила мест v3 работают с любой моделью, в том числе с ботом v2.

models/coop-deberta-v3-ens3-v31 · 24 приказа · 633 строки и 580 seed-команд · три модели · гейт family, порог 0.62 (обучена на RTX 4090, CUDA)

Что изменилось. Архитектура и строки те же, что у v3. Изменился только набор seed-команд: он построен как надстройка над набором v3 (раздел 4.2). Все 383 прежние команды сохранены, две команды про крышу перемечены в `MOVE_TO`, добавлены шаблоны с местами каждого вида и коллауты о состоянии с меткой `NONE`.

Проба «взялась ли починка»	Бот v2	v3	v31
«<место> is smoked» исполняется как приказ	7 из 47	47 из 47	0 из 47
«<место> clear» исполняется как штурм	19 из 30	27 из 30	0 из 30
Приказ идти на крышу без верёвки → <code>RAPPEL</code> (42 строки)	40	18	0
«take <цвет>» → <code>MOVE_TO</code>	0 из 12	—	11 из 12
Обычный приказ «smoke <место>» по-прежнему исполняется	30 из 30	—	30 из 30

Набор v31 писался по этим же пробам, поэтому таблица подтверждает, что правка сработала, но качество не оценивает.

- **На кросс-валидации изменений не найдено.** Разница с v3 по критерию: 0.0 [−6.7, +6.7] на строках с местами и −0.6 [−4.6, +3.3] на старых строках. Против бота v2 на строках с местами: +11.3 [+0.7, +22.7].
- **Цена починки.** «clear main» и голое «clear» модель теперь считает коллаутом и не действует: граница сдвинулась шире, чем хотели. На seed-командах заглавными буквами чужое семейство на 12 из 233 против 8 у v2. Потеря до 5 пунктов на старых строках не исключена.
- **Слепой цифры у v31 нет.** Все её цифры, кроме 80 строк свежего автора из ревью, получены после подгонки. Честную цифру даст только вторая слепая партия строк (`blind/v3b`): скрипты готовы, строк ещё нет.
- **Урок процесса.** Первый вариант набора v31 заново выбирал места для шаблонов и потерял «go to the roof» — то самое, что чинил. Это поймала проверка данных перед обучением: «что из прежнего набора пропало?»
- **Решение владельца.** С 4 октября 2026 v31 — бот по умолчанию: константа `DEFAULT` в `scripts/coop/bots.py` и `COOP_MODEL_DIR` в `CMakeLists.txt`. Если весов v31 на машине нет, инструменты берут бота v2 и сообщают об этом.

Сводная таблица версий

Версия	Папка в models/	Приказов	Строк + seed	Моделей	Гейт и порог	Статус
v1	coop-deberta-v3-base	22	363 + 204	1	top, 0.78	архив
v2	coop-deberta-v3-ens3-v2	24	483 + 233	3	family, 0.58	по умолчанию до 4 октября; запасной
v3	coop-deberta-v3-ens3-v3	24	633 + 383	3	family, 0.60	обучена, по умолчанию не была
v31	coop-deberta-v3-ens3-v31	24	633 + 580	3	family, 0.62	по умолчанию

Качество по версиям

Строки таблицы посчитаны на разных наборах строк, поэтому сравнивать их между собой напрямую нельзя: каждая версия добавляла более трудные строки. Сравнения на одинаковых строках приведены в описании версий выше.

Кросс-валидация по авторам	Строк	Попадание	Чужое семейство	На не-приказ	Критерий
v1: одна модель, без seed-команд	363	75.5	4.8%	13.7%	65.9
v2: ансамбль, гейт family	483	90.5	1.9%	5.3%	86.7
v3: ансамбль, гейт family	633	91.0	2.1%	6.1%	86.9
v31: ансамбль, гейт family	633	91.2	2.4%	7.1%	86.4

8. Уроки проекта

Большинство этих ошибок нашли независимые проверяющие и живые тесты, а не автор изменений. Рабочая привычка проекта: перед тем как делать вывод, попросить коллегу или отдельного агента попытаться его опровергнуть.

№	Что случилось	Как избежать
1	Чекпойнт загрузился в fp16, веса стали NaN, модель отвечала одним классом	Явно задавать float32; проверять NaN после первых шагов
2	Порог подобран на недообученных моделях и вышел слишком низким	Out-of-fold с тем же числом шагов, что у боевой модели
3	Почти-дубли между обучением и тестом завышали цифры	Разбиение по авторам; отдельная метрика на «новых» строках
4	Модель без порога сравнили с моделью с порогом	Сравнивать при одинаковой доле ошибок
5	ModernBERT оценили на чужом рецепте и недооценили на 5–9 пунктов	Подбирать рецепт для каждой модели отдельно
6	«Паритет» двух моделей при интервале ± 9 пунктов	Писать доверительный интервал рядом с каждой разницей
7	Тестовые данные были намеренно без простых формулировок, и бот не понимал «come here»	Seed-команды в обучении; живой тест перед выпуском
8	Seed-команда «come» перебила отрицание: «do not come to me» → FOLLOW_ME	Отрицания в seed-командах и правило отрицания в начале реплики
9	Новая метка сдвинула границы соседних; починка seed-командами (v21) принесла новые ошибки	Правку seed-команд проверять так же, как модель, и до выпуска
10	Seed-команды противоречили спецификации: крыша осталась под RAPPEL	При смене спецификации перечитывать унаследованные seed-команды
11	Шаблон с выборкой покрыл 150 сочетаний из 486, и «take blue» модель не выучила	Каждый шаблон заполнять местами каждого вида
12	Модель исполняла коллауты о состоянии; слепые строки этого не показали	Пробы и ревью в дополнение к слепому тесту; такие коллауты — в NONE
13	Новый набор seed-команд потерял то, что чинил	Правка данных — надстройка над прежним набором, а не новая выборка
14	Правила мест подгонялись под тестовые строки так же, как модель	Слепая цифра только на строках, которых при правке никто не видел
15	Экспорт в ONNX оставил dropout включённым, golden-файл оказался шумом	Явный eval() после экспорта; сверка ONNX с PyTorch до C++

9. Шпаргалка: команды и карта файлов

9.1. Окружение и запуск бота

```
python3.11 -m venv .venv && source .venv/bin/activate
pip install torch "transformers==5.17.0" "tokenizers==0.23.2" \
    onnxruntime onnx scikit-learn numpy sentencepiece
cd scripts/coop
python coop_bot.py # по умолчанию v31; без её весов — v2
python coop_bot.py --model ../../models/coop-deberta-v3-ens3-v2 # другой бот
```

Команды в чате: `/why` включает и выключает отладочную строку, `/t 0.6` меняет порог, `/q` выходит. Версии `transformers` и `tokenizers` — те, на которых всё обучалось и проверялось.

9.2. Обучить и выпустить модель

```
cd scripts/coop
python train_v2.py smoke # минута: устройство работает
COOP_TAG=v31 python train_v2.py final ens3 # 18 обучений, итог в models/
python export_cpp.py ../../models/coop-deberta-v3-ens3-v31 # ONNX, словарь, конфигурация, golden
python check_onnx.py ../../models/coop-deberta-v3-ens3-v31 # ONNX Runtime против PyTorch
python ../../cpp/coop_intent/tools/gen_tests.py ../../models/coop-deberta-v3-ens3-v31 # тесты для C++
```

9.3. Оценить модель

```
COOP_TAG=v31 python train_v2.py cv base --held r6 --results results_v31_probs_r6.json # и так же cs, stt
COOP_TAG=v31 python eval_v3.py > eval_v31.log # нужен scikit-learn
COOP_TAG=v31 python probe_v3.py ../../models/coop-deberta-v3-ens3-v31 \
    ../../models/coop-deberta-v3-ens3-v2 results_v31_probe.json > probe_v31.log # пробы
```

9.4. Собрать и проверить C++-движок (Windows)

```
cd cpp/coop_intent
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Release
build\Release\coop_cli.exe # чат, как coop_bot.py
build\Release\coop_cli.exe --golden # и остальные: --tokenizer-tests --location-tests
# --gate-tests --decide-tests --dialogue --bench
```

`COOP_MODEL_DIR` в CMake — переменная кэша: если поменять бота по умолчанию, уже сконфигурированная папка `build` останется на прежнем. Нужно передать `-DCOOP_MODEL_DIR=...` или удалить `build`.

9.5. Поменялся только словарь мест, правила или регэкспы

Переобучать модель и заново экспортировать ONNX не нужно. Для каждой папки бота:

```
python scripts/coop/locations.py --dev # регрессионные строки разработчика
python scripts/coop/export_cpp.py models/<бот> --config-only # только конфигурация и словарь
python cpp/coop_intent/tools/gen_tests.py models/<бот> # --no-model, если весов бота нет
```

9.6. Карта файлов

Файл или папка	Что это
HANDOFF.md	Текущее состояние проекта, пошаговые команды, открытые решения
COOP-BOT.md	Исследование целиком: протокол, все таблицы, оговорки
DEBERTA-BOT.md	Разбор «как сделан бот» с ловушками

Файл или папка	Что это
<code>cpp/coop_intent/README.md</code>	C++-движок, проверки, замеры, план переноса в UE5
<code>scripts/coop/train_v2.py</code>	Обучение: <code>smoke</code> , <code>cv</code> , <code>final</code>
<code>scripts/coop/coop_v2.py</code>	Данные, семейства, гейты, подсчёт метрик, подбор порога
<code>scripts/coop/coop_bot.py</code>	Python-бот: классификация, гейт, логика решения, чат
<code>scripts/coop/bots.py</code>	Единственное место, где назван бот по умолчанию
<code>scripts/coop/timing_rule.py</code>	Регэкспы <code>ON_SIGNAL</code> и <code>OTHER</code>
<code>scripts/coop/locations.json</code> , <code>locations.py</code>	Словарь мест и правила сопоставителя
<code>scripts/coop/seed_commands_v*.json</code>	Наборы seed-команд; <code>make_seeds_v31.py</code> строит последний
<code>scripts/coop/blind/</code> , <code>annot/</code>	Слепые строки авторов и их разметка
<code>scripts/coop/export_cpp.py</code> , <code>check_onnx.py</code>	Экспорт в ONNX и его проверка
<code>scripts/coop/frozen.txt</code>	Журнал заморозки: хэши и порядок событий
<code>models/coop-deberta-v3-*/</code>	Обученные боты: конфигурация в git, веса — вне git

9.7. Что дальше

- **Вторая слепая партия строк** (`blind/v3b`). Это единственный способ получить честную цифру для правил мест v3 и модели v31. В партию стоит включить коллауты о состоянии.
- **Плагин UE5**. Модель через NNE; регэкспы через ICU по ASCII-«тени» строки; места — GameplayTags на акторах уровня; договор с планировщиком о ролях и пометах.
- **Открытые решения владельца**. `WAIT` для отложенного приказа — пауза или отмена. Приводить ли реплику к строчным буквам перед классификатором. Как читать «on two»: счёт сигнала или этаж.
- **Настоящие данные**. Записывать реплики игроков, размечать в первую очередь те, где бот переспросил или ошибся, и прогонять записи через тот же движок распознавания речи, что будет в игре.